Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Структури даних та алгоритми
Лабораторна робота №4

«Алгоритми двійкового пошуку»

Виконав:
студент групи ІО-61
Астаф'єв Олександр Владиславович
Номер у списку групи: 2
Перевірив:
Русінов Володимир Володимирович

Київ-2026

Мета лабораторної роботи:

Засвоєння теоретичного матеріалу та набуття практичних навичок розв'язання задачі пошуку заданої категорії елементів за допомогою різних алгоритмів методу двійкового пошуку у двовимірних масивах.

Постановка задачі:

1. Написати програму розв'язання задачі пошуку (за варіантом) у двовимірному масиві (матриці) методом двійкового пошуку. Конкретний алгоритм двійкового пошуку (№1 або №2) задається варіантом завдання.
2. Розміри матриці m та n взяти самостійно в межах від 7 до 10.
3. При тестуванні програми підбирати такі вхідні набори значень матриці, щоб можна було легко відстежити коректність виконання пошуку, і щоб ця коректність була протестована для всіх можливих випадків. З метою тестування дозволяється використовувати матриці меншого розміру.

2 Задано матрицю дійсних чисел A[m,n]. Окремо у кожному стовпчику матриці визначити присутність заданого дійсного числа X і його місцезнаходження (координати) методом двійкового пошуку (**Алгоритм №2**), якщо елементи кожного стовпчика окремо впорядковані **за незбільшенням**.

Текст програми:

```
#include <stdio.h>


#define M 9

#define N 9


int binary_search_2(float A[M][N], int col, float x)

{

   int L = 0, R = M - 1, i;


   while (L < R)

   {

      i = (L + R) / 2;

      if (A[i][col] > x)
```

```
            L = i + 1;
        else
            R = i;
    }
    if (A[R][col] == x)
        return R;
    return -1;
}

int main()
{
    float A[M][N] = {
        {  35.0,   30.0,   32.5,   31.0,   34.0,   33.5,   35.0,   29.5,   32.0},
        {  35.0,   26.5,   27.0,   25.5,   29.0,   28.0,   30.5,   29.5,   26.0},
        {  35.0,   26.5,   21.5,   25.5,   24.0,   28.0,   23.5,   29.5,   19.5},
        {  28.5,   26.5,   16.0,   20.0,   24.0,   28.0,   18.0,   29.5,   19.5},
        {  22.0,   26.5,   16.0,   14.5,   24.0,   21.0,   18.0,   20.5,   19.5},
        {  22.0,   19.0,   16.0,    9.0,   17.5,   21.0,   12.0,   14.0,   19.5},
        {  15.5,   12.5,   16.0,    3.5,   11.0,   21.0,   12.0,    7.5,   10.0},
        {   9.0,    5.0,    8.5,    3.5,    4.5,    6.0,   12.0,    2.0,    4.0},
        {   2.5,   -3.0,    1.0,    3.5,   -2.0,    0.0,   12.0,   -1.5,    0.0}
    };
    float x;
    int i, j, r, any = 0;
```

```
printf("Matrix A[%d,%d]:\n", M, N);
for (i = 0; i < M; i++)
{
    for (j = 0; j < N; j++)
        printf("%6.1f", A[i][j]);
    printf("\n");
}

printf("\nEnter X: ");
scanf("%f", &x);
printf("\n");

for (j = 0; j < N; j++)
{
    r = binary_search_2(A, j, x);
    if (r != -1)
    {
        printf("A[%d][%d] = %.1f;\n", r + 1, j + 1, A[r][j]);
        any = 1;
    }
}
if (!any)
    printf("Number %.2f not found.\n", x);

return 0;
```

}

Результати програми:

```
PS D:\Astafiev\АСД\sda-astafiev\Lab4> .\task.exe
Matrix A[9,9]:
  35.0  30.0  32.5  31.0  34.0  33.5  35.0  29.5  32.0
  35.0  26.5  27.0  25.5  29.0  28.0  30.5  29.5  26.0
  35.0  26.5  21.5  25.5  24.0  28.0  23.5  29.5  19.5
  28.5  26.5  16.0  20.0  24.0  28.0  18.0  29.5  19.5
  22.0  26.5  16.0  14.5  24.0  21.0  18.0  20.5  19.5
  22.0  19.0  16.0   9.0  17.5  21.0  12.0  14.0  19.5
  15.5  12.5  16.0   3.5  11.0  21.0  12.0   7.5  10.0
   9.0   5.0   8.5   3.5   4.5   6.0  12.0   2.0   4.0
   2.5  -3.0   1.0   3.5  -2.0   0.0  12.0  -1.5   0.0

Enter X: 32.0

A[1][9] = 32.0;
```

```
PS D:\Astafiev\АСД\sda-astafiev\Lab4> .\task.exe
Matrix A[9,9]:
  35.0  30.0  32.5  31.0  34.0  33.5  35.0  29.5  32.0
  35.0  26.5  27.0  25.5  29.0  28.0  30.5  29.5  26.0
  35.0  26.5  21.5  25.5  24.0  28.0  23.5  29.5  19.5
  28.5  26.5  16.0  20.0  24.0  28.0  18.0  29.5  19.5
  22.0  26.5  16.0  14.5  24.0  21.0  18.0  20.5  19.5
  22.0  19.0  16.0   9.0  17.5  21.0  12.0  14.0  19.5
  15.5  12.5  16.0   3.5  11.0  21.0  12.0   7.5  10.0
   9.0   5.0   8.5   3.5   4.5   6.0  12.0   2.0   4.0
   2.5  -3.0   1.0   3.5  -2.0   0.0  12.0  -1.5   0.0

Enter X: 9.0

A[8][1] = 9.0;
A[6][4] = 9.0;
```

```
PS D:\Astafiev\АСД\sda-astafiev\Lab4> .\task.exe
Matrix A[9,9]:
  35.0  30.0  32.5  31.0  34.0  33.5  35.0  29.5  32.0
  35.0  26.5  27.0  25.5  29.0  28.0  30.5  29.5  26.0
  35.0  26.5  21.5  25.5  24.0  28.0  23.5  29.5  19.5
  28.5  26.5  16.0  20.0  24.0  28.0  18.0  29.5  19.5
  22.0  26.5  16.0  14.5  24.0  21.0  18.0  20.5  19.5
  22.0  19.0  16.0   9.0  17.5  21.0  12.0  14.0  19.5
  15.5  12.5  16.0   3.5  11.0  21.0  12.0   7.5  10.0
   9.0   5.0   8.5   3.5   4.5   6.0  12.0   2.0   4.0
   2.5  -3.0   1.0   3.5  -2.0   0.0  12.0  -1.5   0.0

Enter X: 7.0

Number 7.00 not found.
```

Висновки за результатами лабораторної роботи:

У ході лабораторної роботи було реалізовано пошук у матриці 9×9, кожен стовпчик якої впорядкований за незбільшенням. За варіантом 2 використано

алгоритм двійкового пошуку №2. Він застосовується окремо до кожного стовпчика «на місці», без додаткових масивів.

Програму перевірено на трьох наборах даних:

1. **X = 32.0.** Знайдено один елемент, A[1][9], тобто верхній елемент крайнього правого стовпчика (початок області пошуку). В інших стовпчиках значення немає. Близькі 32.5 (стовпчик 3) і 31.0 (стовпчик 4) програма правильно не прийняла за збіг.
2. **X = 9.0.** Знайдено два елементи в різних стовпчиках: A[8][1] у нижній частині стовпчика 1 і A[6][4] ближче до середини стовпчика 4. Для кожного стовпчика виводиться не більше одного результату.
3. **X = 7.0.** Значення в матриці відсутнє, програма вивела «Number 7.00 not found.». Число потрапляє між сусідніми елементами більшості стовпчиків, а в стовпчику 7 (усі елементи ≥ 12.0) воно менше за всі. Хибних спрацювань не було, тож алгоритм коректно працює в усіх перевірених випадках: елемент на початку, посередині, в кінці стовпчика та відсутній.

Стовпчик має 9 елементів, тому алгоритм виконує 3–4 ітерації. На всю матрицю це 27–36 порівнянь. Лінійний пошук у найгіршому випадку (елемент відсутній) потребує 9 порівнянь на стовпчик, тобто 81 на матрицю. Виграш становить 2–3 рази, і він зростає зі збільшенням розміру стовпчика, бо O(log n) проти O(n). У матриці є серії однакових значень, наприклад 26.5 у рядках 2–5 стовпчика 2 і 19.5 у рядках 3–6 стовпчика 9. Алгоритм №2 знаходить найлівіший (найвищий) елемент серії, як і лінійний пошук: для X = 26.5 це рядок 2, для X = 19.5 це рядок 3. Алгоритм №1 зупиняється на першому ж збігу з серединою діапазону й повертає випадковий елемент серії (на цих даних рядок 5 в обох випадках). Ціна детермінованості в тому, що №2 не завершується раніше і завжди виконує 3–4 ітерації, тоді як №1 може знайти елемент за 1 крок.